



# Matrizes

:::: Introdução a Algoritmos



# Agora, é sua vez!

Imagine que estamos desenvolvendo um jogo. Um jogador realizou 5 jogadas, e em cada jogada ele fez uma pontuação. Crie um programa que:

- leia as pontuações das 5 jogadas
- armazene esses valores
- calcule a média das pontuações
- exiba:
  - uma lista com as pontuações acima da média
  - uma lista com as pontuações abaixo da média

# Pensando no Problema

## Entrada:

- pontuação de cada jogada

## Processamento:

- armazenar os valores
- calcular a média
- separar os valores em dois grupos

## Saída:

- lista de valores acima da média
- lista de valores abaixo da média

# Casos de Teste

| Entrada             | Saída Esperada         |
|---------------------|------------------------|
| 10, 20, 30, 40, 50  | [40, 50] [10, 20]      |
| 5, 5, 5, 5, 5       | [ ] [ ]                |
| 100, 10, 90, 20, 80 | [100, 90, 80] [10, 20] |

# Implementação

```
● ● ●  
  
pontuacoes = []  
  
for i in range(5):  
    pontos = int(input("Pontuação: "))  
    pontuacoes.append(pontos)  
  
media = sum(pontuacoes) / len(pontuacoes)  
  
acima = []  
abaixo = []  
  
for p in pontuacoes:  
    if p > media:  
        acima.append(p)  
    elif p < media:  
        abaixo.append(p)  
  
print(acima)  
print(abaixo)
```

# Explorando a Solução

```

pontuacoes = []

for i in range(5):
    pontos = int(input("Pontuação: "))
    pontuacoes.append(pontos)

media = sum(pontuacoes) / len(pontuacoes)

acima = []
abaixo = []

for p in pontuacoes:
    if p > media:
        acima.append(p)
    elif p < media:
        abaixo.append(p)

print(acima)
print(abaixo)

```

Note que:

- armazenamos dados em uma lista
- calculamos a média
- percorremos os valores
- organizamos os resultados em novas listas

# Novo Cenário

Vamos Adaptar o programa anterior para:

- ler as pontuações de 3 jogadores
- para cada jogador:
  - armazenar suas pontuações
  - calcular a média
  - exibir:
    - pontuações acima da média
    - pontuações abaixo da média

# Pensando no Problema

## Entrada:

- pontuações de vários jogadores

## Processamento:

- repetir o processo para cada jogador
- armazenar e analisar os dados

## Saída:

- resultado para cada jogador



# Uma Possível Abordagem

Uma forma de resolver seria:



```
# Jogador 1  
pontuacoes1 = []
```

```
# Jogador 2  
pontuacoes2 = []
```

```
# Jogador 3  
pontuacoes3 = []
```

# Problema da Abordagem



```
# Jogador 1  
pontuacoes1 = []  
  
# Jogador 2  
pontuacoes2 = []  
  
# Jogador 3  
pontuacoes3 = []
```

Note que essa solução:

- funciona para poucos jogadores
- exige repetir código
- não escala bem

👉 E se fossem 10 jogadores?

👉 E se fossem 100?

# Reflexão



```
# Jogador 1  
pontuacoes1 = []  
  
# Jogador 2  
pontuacoes2 = []  
  
# Jogador 3  
pontuacoes3 = []
```

Temos:

- vários jogadores
- cada jogador com várias pontuações

💬 Como podemos organizar esses dados em uma única estrutura?

# Construindo a Ideia

Uma possibilidade seria agrupar as listas:

```
● ● ●  
  
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]
```

# Visualizando os Dados

Cada posição representa:

- um jogador
- com sua lista de pontuações



```
pontuacoes = [  
    [10, 20, 30, 40, 50], → Jogador 1  
    [15, 25, 35, 45, 55], → Jogador 2  
    [5, 10, 15, 20, 25]  → Jogador 3  
]
```

# Explorando a Estrutura



```
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]
```

Observe que:

- temos uma lista principal
- cada posição guarda outra lista

👉 Estamos trabalhando com uma coleção de coleções

# O que estamos construindo?

Até agora, criamos uma lista com várias listas dentro. Essa estrutura tem um nome:

Matriz

Uma matriz é uma estrutura organizada em **linhas** e **colunas**. Exemplo:

$$\begin{Bmatrix} 10 & 20 & 30 & 40 & 50 \\ 15 & 25 & 35 & 45 & 55 \\ 5 & 10 & 15 & 20 & 25 \end{Bmatrix}$$

# Pensando como uma Tabela

Podemos visualizar essa estrutura assim:

|   | 0  | 1  | 2  | 3  | 4  |
|---|----|----|----|----|----|
| 0 | 10 | 20 | 30 | 40 | 50 |
| 1 | 15 | 25 | 35 | 45 | 55 |
| 2 | 5  | 10 | 15 | 20 | 25 |

Temos:

- linhas (horizontal)
- colunas (vertical)

👉 Isso é uma estrutura **bidimensional**



# Por que usar matrizes?

Muitas situações do mundo real usam esse formato. Exemplos:

- planilhas (Excel)
- jogos (tabuleiro, mapas)
- imagens (pixels)
- notas de alunos por disciplina

Em algoritmos, usamos matrizes quando precisamos organizar dados em duas dimensões!

# Ligando a Ideia ao Código

No código, uma matriz é representado por uma **lista de listas**!

Ou seja:

- cada linha → uma lista
- a matriz → lista principal

Veja:

```
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]
```



|   | 0  | 1  | 2  | 3  | 4  |
|---|----|----|----|----|----|
| 0 | 10 | 20 | 30 | 40 | 50 |
| 1 | 15 | 25 | 35 | 45 | 55 |
| 2 | 5  | 10 | 15 | 20 | 25 |

# Índices em uma Matriz

Para acessar um elemento usamos:

```
matriz[i][j]
```

Onde:

- $i \rightarrow$  linha
- $j \rightarrow$  coluna

👉 Primeiro escolhemos a linha

👉 Depois a coluna

# Como Pensar em *i* e *j*?

Pense sempre assim:

```
matriz[i][j]
```

- *i* → “qual linha?”
- *j* → “qual coluna?”

Exemplo:

- `matriz[1][2]` → linha 1, coluna 2

# Acesso aos Dados

Podemos acessar os valores assim:

```
● ● ●  
  
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]  
  
print(pontuacoes[0])           # Jogador 1  
print(pontuacoes[0][0])       # primeira jogada  
print(pontuacoes[1][2])       # jogador 2, jogada 3
```

# Visualizando a Matriz

```
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]
```

|         |   | Pontuações |    |    |    |    |
|---------|---|------------|----|----|----|----|
|         |   | 0          | 1  | 2  | 3  | 4  |
| Jogador | 0 | 10         | 20 | 30 | 40 | 50 |
|         | 1 | 15         | 25 | 35 | 45 | 55 |
|         | 2 | 5          | 10 | 15 | 20 | 25 |

## Exemplos

- `pontuacoes[0][0]` → ?
- `pontuacoes[1][3]` → ?
- `pontuacoes[2][4]` → ?

# Observação Importante

Assim como nas listas:

- os índices começam em 0
- o primeiro índice indica o “grupo”
- o segundo índice indica o valor dentro do grupo

👉 Estamos acessando dados em **dois níveis**

# Percorrendo a Estrutura

Para percorrer uma lista, fazemos:

```
for p in pontuacoes:  
    print(p)
```

Isso funciona porque estamos percorrendo uma coleção de valores!



# Aplicando na Nova Estrutura

O que acontece se fizermos:

```
● ● ●  
  
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]  
  
for jogador in pontuacoes:  
    print(jogador)
```

# Aplicando na Nova Estrutura

O que acontece se fizermos:

```
● ● ●  
  
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]  
  
for jogador in pontuacoes:  
    print(jogador)
```



```
● ● ●  
  
[10, 20, 30, 40, 50]  
[15, 25, 35, 45, 55]  
[5, 10, 15, 20, 25]
```

Note que estamos percorrendo jogador por jogador. Ou seja, lista por lista!

# Como acessar cada pontuação?



```
for jogador in pontuacoes:  
    for pontos in jogador:  
        print(pontos)
```

Entendendo o que acontece:

- o primeiro for percorre os jogadores
- o segundo for percorre as pontuações

Ou seja, estamos percorrendo dois níveis de dados!

# Padrão Importante

**Lista:**



```
for valor in lista:
```

**Matriz:**



```
for grupo in estrutura:  
    for valor in grupo:
```

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices

```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |



# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
|       | 0  | 1  |
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices

```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Percorrendo com Índices



```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):  
        print(pontuacoes[i][j])
```

| i \ j | 0  | 1  |
|-------|----|----|
| 0     | 10 | 20 |
| 1     | 15 | 25 |

# Comparação Importante

Sem índice:

```
for jogador in pontuacoes:  
    for pontos in jogador:
```

Com índice:

```
for i in range(len(pontuacoes)):  
    for j in range(len(pontuacoes[i])):
```

Usamos índices quando precisamos saber a posição do elemento!



Dica: A grande maioria dos algoritmos clássicos com matrizes trabalham com índices. Logo, acostume-se a manipular  $i$  e  $j$ .



# Novo Problema

Até agora usamos uma estrutura pronta. Agora queremos:

- ler as pontuações de 3 jogadores
- cada jogador com 5 jogadas
- armazenar todos os dados

# Relembrando Listas

Para criar uma lista, fazemos:

```
valores = []
```

E adicionamos elementos com:

```
valores.append(x)
```

Como podemos fazer isso com várias listas?

# Construindo Passo a Passo

Para um jogador:



```
jogador = []  
  
for j in range(5):  
    pontos = int(input("Pontuação: "))  
    jogador.append(pontos)
```

# Construindo a Estrutura Completa

```
● ● ●  
  
pontuacoes = []  
  
for i in range(3):  
    jogador = []  
  
    for j in range(5):  
        pontos = int(input("Pontuação: "))  
        jogador.append(pontos)  
  
    pontuacoes.append(jogador)
```

# Entendo o que foi feito

```
● ● ●  
  
pontuacoes = []  
  
for i in range(3):  
    jogador = []  
  
    for j in range(5):  
        pontos = int(input("Pontuação: "))  
        jogador.append(pontos)  
  
    pontuacoes.append(jogador)
```

1. criamos uma lista principal
2. para cada jogador, criamos uma nova lista
3. adicionamos as pontuações nessa lista
4. inserimos a lista do jogador na estrutura principal

# Estrutura Final

Após a execução, teremos algo como:

```
● ● ●  
  
pontuacoes = [  
    [10, 20, 30, 40, 50],  
    [15, 25, 35, 45, 55],  
    [5, 10, 15, 20, 25]  
]
```

# Desafio 1: Médias por Jogador

Imagine que estamos desenvolvendo um sistema para analisar o desempenho de jogadores em um campeonato. As pontuações dos jogadores foram armazenadas em uma matriz, onde cada linha representa um jogador e cada coluna representa uma partida. Crie um programa que percorra essa matriz e calcule a média de pontuação de cada jogador.

Ao final, exiba o resultado no seguinte formato:

- Media do jogador 1: X pontos
- Media do jogador 2: X pontos

# Pensando no Problema

## Entrada:

- matriz de pontuações

## Processamento:

- percorrer cada jogador (linha)
- somar suas pontuações
- calcular a média

## Saída:

- média de cada jogador formatada



# Casos de Teste

| Entrada                             | Saída Esperada                                               |
|-------------------------------------|--------------------------------------------------------------|
| <code>[[[]]]</code>                 | Média do Jogador 1: 0 pontos                                 |
| <code>[[1, 2, 3], [4, 5, 6]]</code> | Média do Jogador 1: 2 pontos<br>Média do Jogador 2: 5 pontos |

# Implementação

```

pontuacoes = [
    [10, 20, 30],
    [40, 50, 60],
    [15, 15, 15]
]

for i in range(len(pontuacoes)):
    soma = 0

    for j in range(len(pontuacoes[i])):
        soma += pontuacoes[i][j]

    media = soma / len(pontuacoes[i])

    print(f"Media do jogador {i+1}: {media} pontos")

```

# Desafio 2: Comparação com Média Geral

Agora queremos dar um passo além na análise. Utilizando a mesma matriz de pontuações, crie um programa que:

- calcule a média de pontuação de cada jogador
- calcule a média geral considerando todos os jogadores
- exiba quais jogadores tiveram média acima da média geral

# Pensando no Problema

## Entrada:

- matriz de pontuações

## Processamento:

- calcular média de cada jogador
- armazenar essas médias
- calcular média geral
- comparar

## Saída:

- jogadores acima da média geral

# Casos de Teste

| Entrada                             | Saída Esperada                             |
|-------------------------------------|--------------------------------------------|
| <code>[[[]]]</code>                 | Jogadores acima da média: <code>[]</code>  |
| <code>[[1, 2, 3], [4, 5, 6]]</code> | Jogadores acima da média: <code>[2]</code> |

# Implementação

```
● ● ●  
  
pontuacoes = [  
    [10, 20, 30],  
    [40, 50, 60],  
    [15, 15, 15]  
]  
  
medias = []  
for i in range(len(pontuacoes)):  
    soma = 0  
  
    for j in range(len(pontuacoes[i])):  
        soma += pontuacoes[i][j]  
  
    media = soma / len(pontuacoes[i])  
    medias.append(media)  
  
media_geral = sum(medias) / len(medias)  
  
resultado = []  
for i in range(len(medias)):  
    if medias[i] > media_geral:  
        resultado.append(i + 1)  
  
print("Jogadores acima da média:", resultado)
```

# Desafio 3: Mapa de Jogo

Em um jogo, o mapa é representado por uma matriz de números. Cada posição contém a quantidade de pontos que o jogador pode coletar naquele local.

O jogador só pode se mover para a direita ou para baixo, começando no canto superior esquerdo e terminando no canto inferior direito.

Crie um programa que determine o maior número de pontos que o jogador pode coletar nesse caminho.

# Pensando no Problema

## Entrada:

- matriz de pontos

## Processamento:

- explorar caminhos possíveis
- somar valores
- escolher o melhor caminho

## Saída:

- maior soma possível



# Casos de Teste

| Entrada                                   | Saída Esperada |
|-------------------------------------------|----------------|
| [[[]]]                                    | 0              |
| [[5, 1, 3],<br>[2, 10, 1],<br>[1, 1, 20]] | 38             |

# Implementação

```
def maior_caminho(matriz):  
    i = 0  
    j = 0  
    soma = matriz[0][0]  
  
    while i < len(matriz) - 1 or j < len(matriz[0]) - 1:  
        if i == len(matriz) - 1:  
            j += 1  
        elif j == len(matriz[0]) - 1:  
            i += 1  
        else:  
            if matriz[i+1][j] > matriz[i][j+1]:  
                i += 1  
            else:  
                j += 1  
  
        soma += matriz[i][j]  
  
    return soma  
  
matriz = [[5, 1, 3],  
          [2, 10, 1],  
          [1, 1, 20]]  
  
print(maior_caminho(matriz))
```

# Exercícios de Fixação

1. Dada uma matriz, exiba todos os elementos.
2. Exiba apenas os elementos da primeira linha.
3. Exiba apenas os elementos da primeira coluna.
4. Calcule a soma de todos os elementos da matriz.
5. Calcule a média de cada linha da matriz.
6. Encontre o maior valor da matriz.
7. Conte quantos números são pares.
8. Exiba apenas os valores maiores que um número informado pelo usuário.

# Exercícios de Fixação

1. Crie uma matriz identidade (1 na diagonal, 0 no resto).
2. Verifique se uma matriz é quadrada.
3. Some todos os elementos da diagonal principal.
4. Inverta os elementos de cada linha.
5. Encontre a linha com maior soma.
6. Encontre a coluna com maior soma.
7. Verifique se todos os elementos de uma linha são iguais.
8. Conte quantos elementos estão acima da média da matriz.



**PUC Minas**

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.

Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.